אוניברסיטת בן-גוריון בנגב
Ben-Gurion University of the Negev

Applied Deep Learning

Omri Azencot

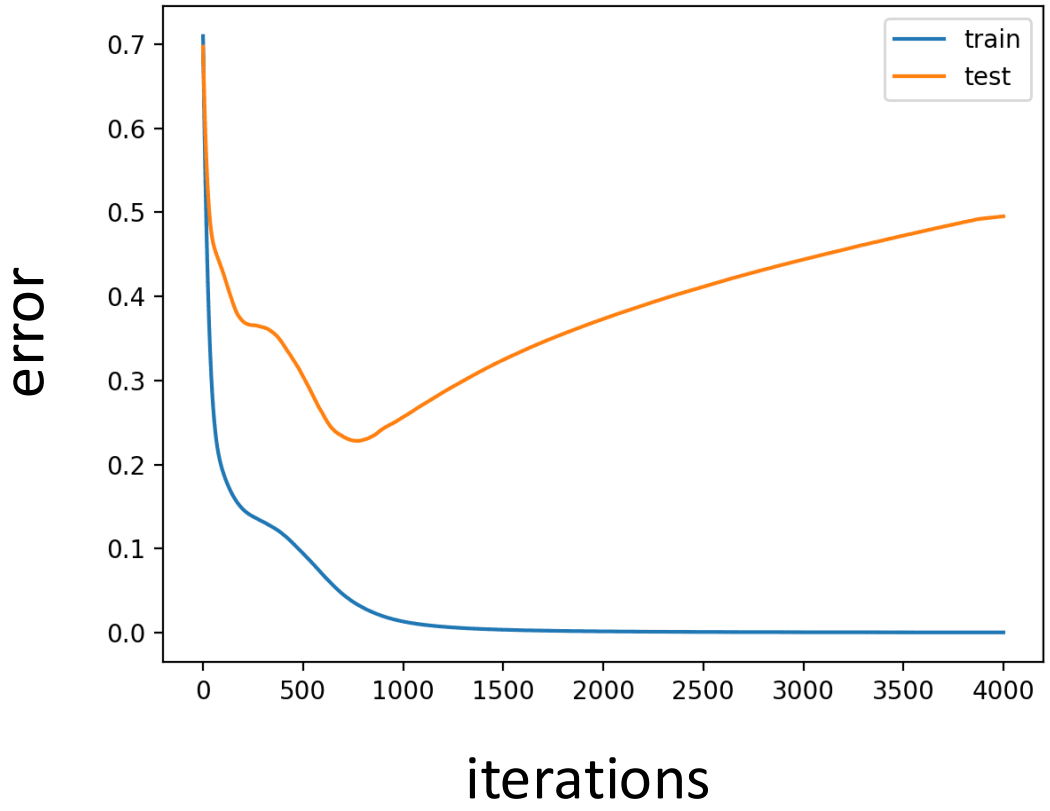
The Stein Faculty of Computer and Information Science

Ben-Gurion University of the Negev

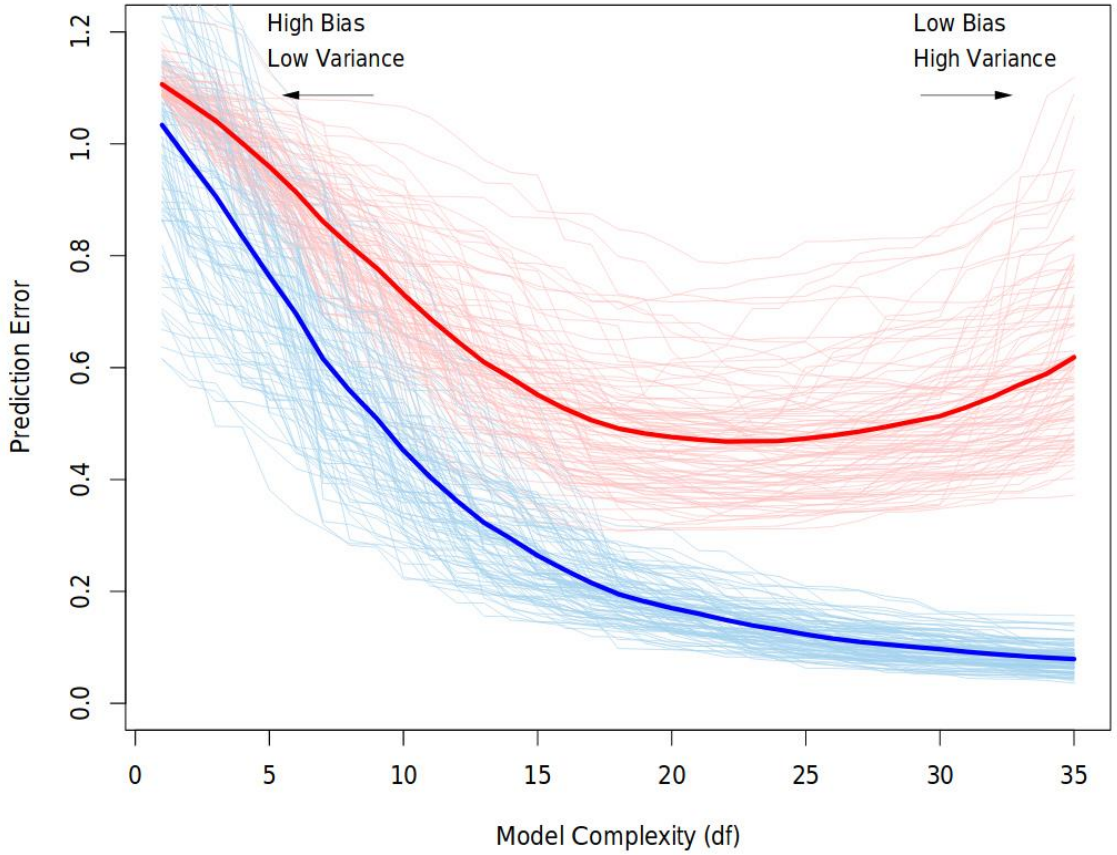
Deep Learning Regularization

Regularization

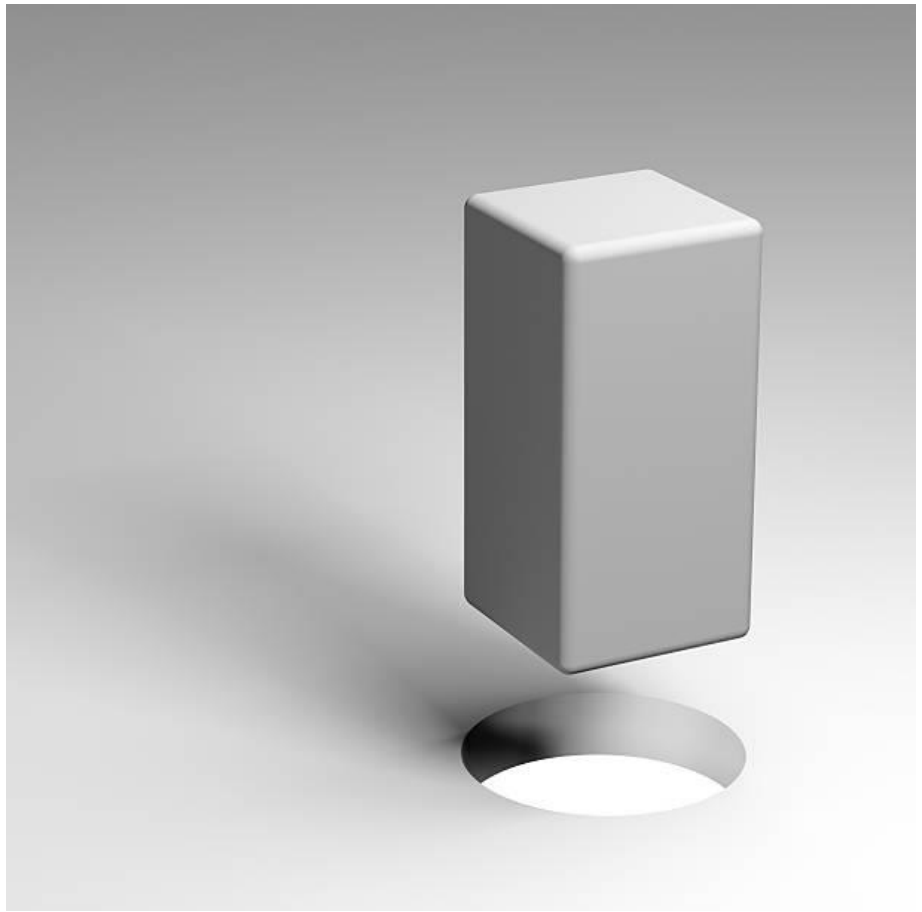
Modify a learning algorithm to reduce generalization error



An Abstract Example

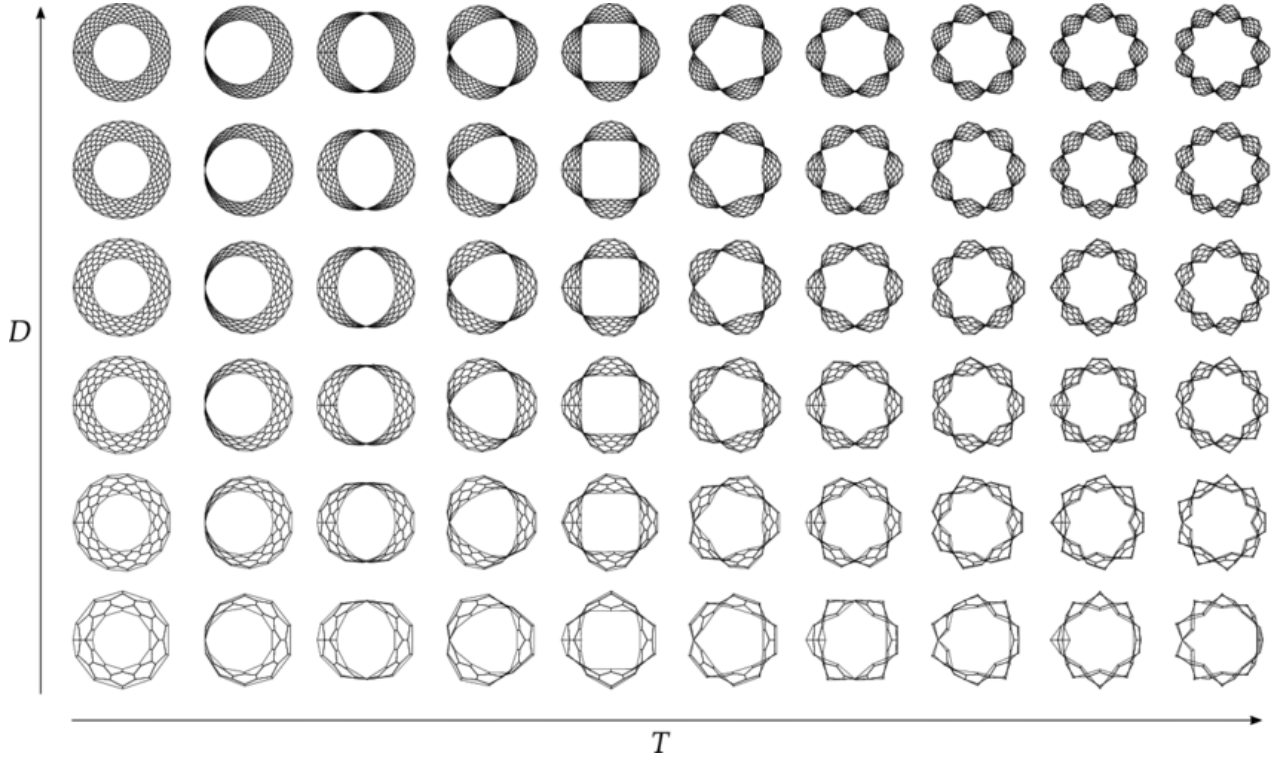


Choosing a Model Family



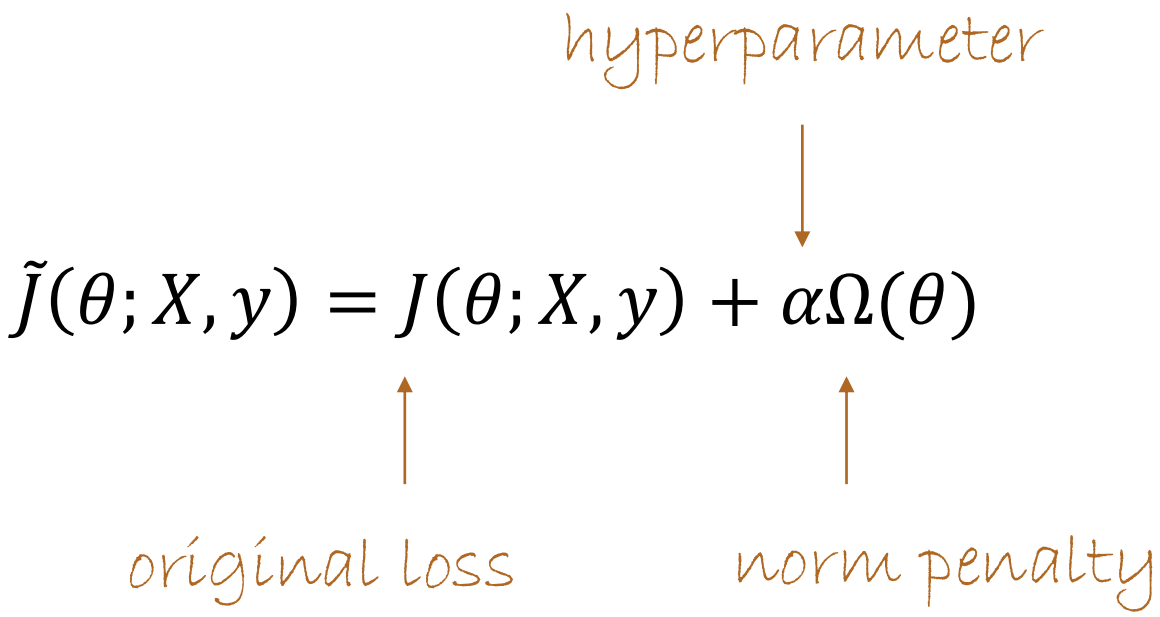
Parameter Norm Penalties

Often, we want to limit the capacity of a model



Parameter Norm Penalties

Introduce a parameter norm penalty



Example: L^2 Parameter Regularization

Drive the weights closer to the origin

no bias weights

$$\Omega(\theta) = \frac{1}{2}|\omega|_2^2$$

weight decay
ridge regression

Weight Decay: Local Analysis

The objective function

$$\tilde{J}(\omega; X, y) = \frac{\alpha}{2}\omega^T\omega + J(\omega; X, y)$$

What happens globally?

The gradient step is

$$\omega \leftarrow (1 - \epsilon\alpha)\omega - \epsilon\nabla_\omega J(\omega; X, y)$$

→ locally: shrink the weight vector by a constant factor

Weight Decay: Global Analysis

Simplify the objective function

$$\hat{J}(\omega) = J(\omega^*) + \frac{1}{2}(\omega - \omega^*)^T H(\omega - \omega^*)$$

Hessian positive semidefinite

The minimum of regularized version

$$\tilde{\omega} = (H + \alpha I)^{-1} H\omega^*$$

What happens if α grows?

Weight Decay: Global Analysis

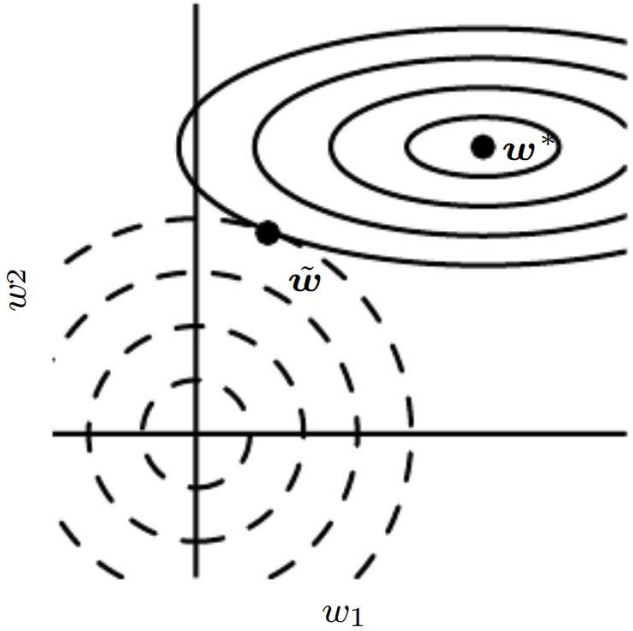
Using $H = Q\Lambda Q^T$, we obtain

$$\tilde{\omega} = Q(\Lambda + \alpha I)^{-1}\Lambda Q^T \omega^*$$

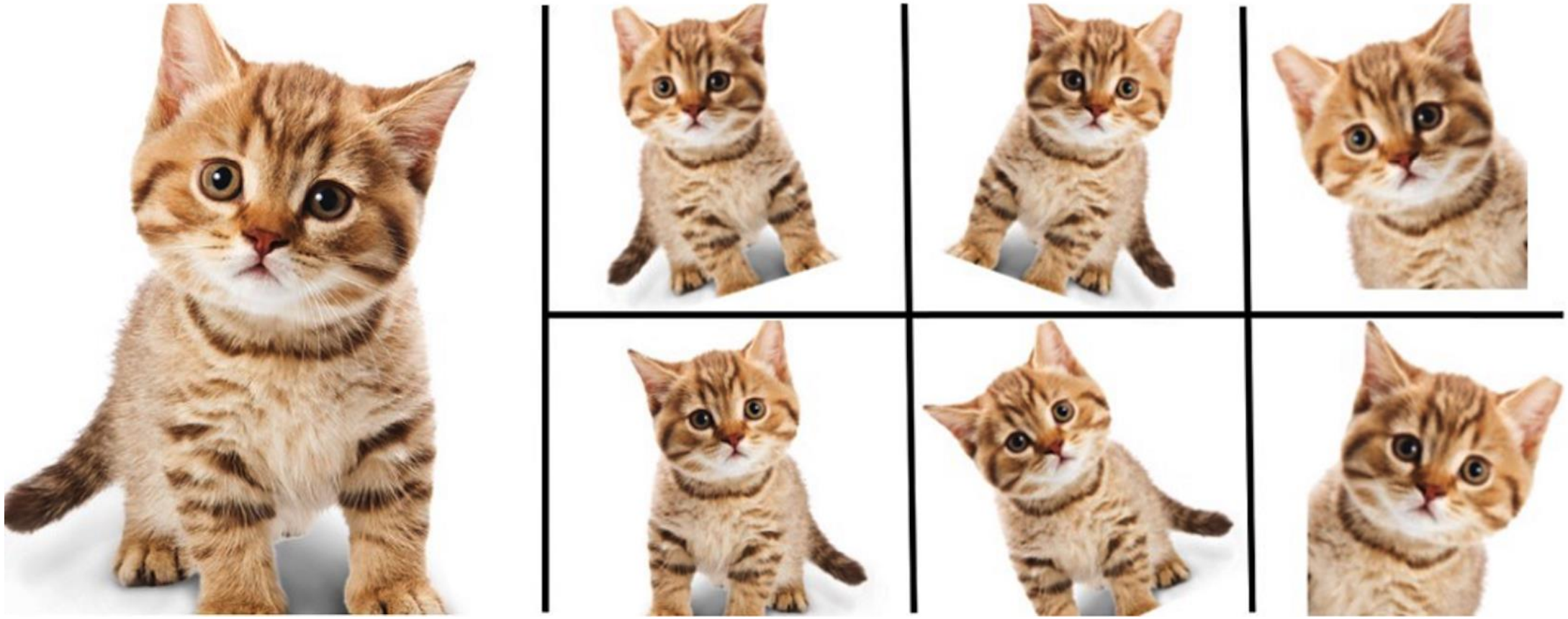
Namely, rescale eigenvectors by

$$\frac{\lambda_i}{\lambda_i + \alpha}$$

Large effect when $\lambda_i \ll \alpha$



Data Augmentation



Empirical vs. Vicinal Risk Minimization

Empirical Risk Minimization

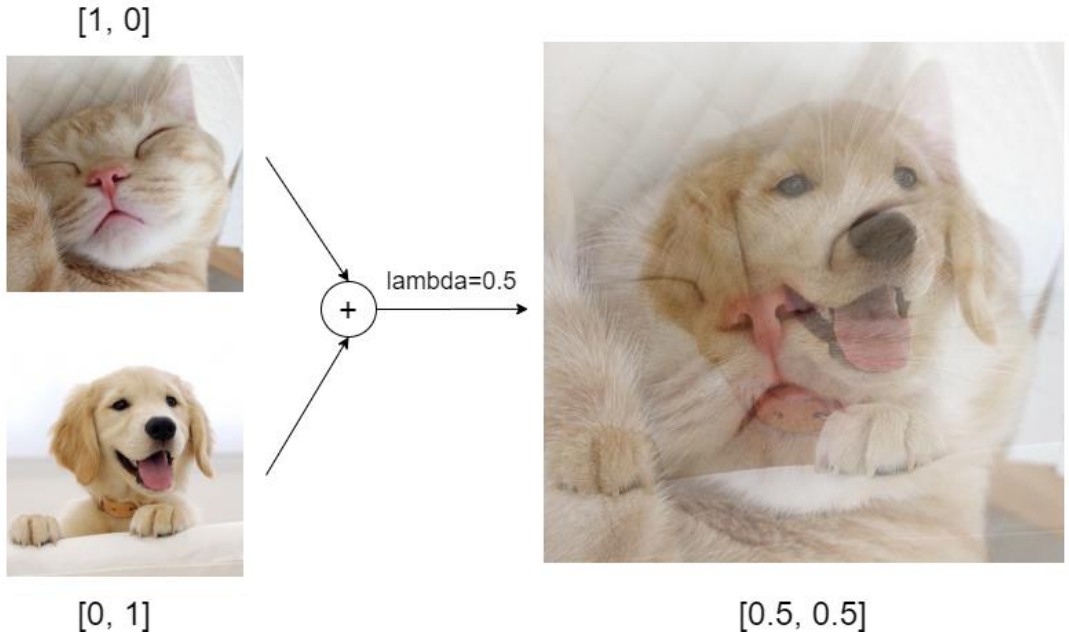
$$\mathrm{d}P_{\mathrm{emp}}(x,y) = \frac{1}{n}\sum_i \delta_{x_i}(x)\delta_{y_i}(y)$$ ← x_i, y_i are in train set

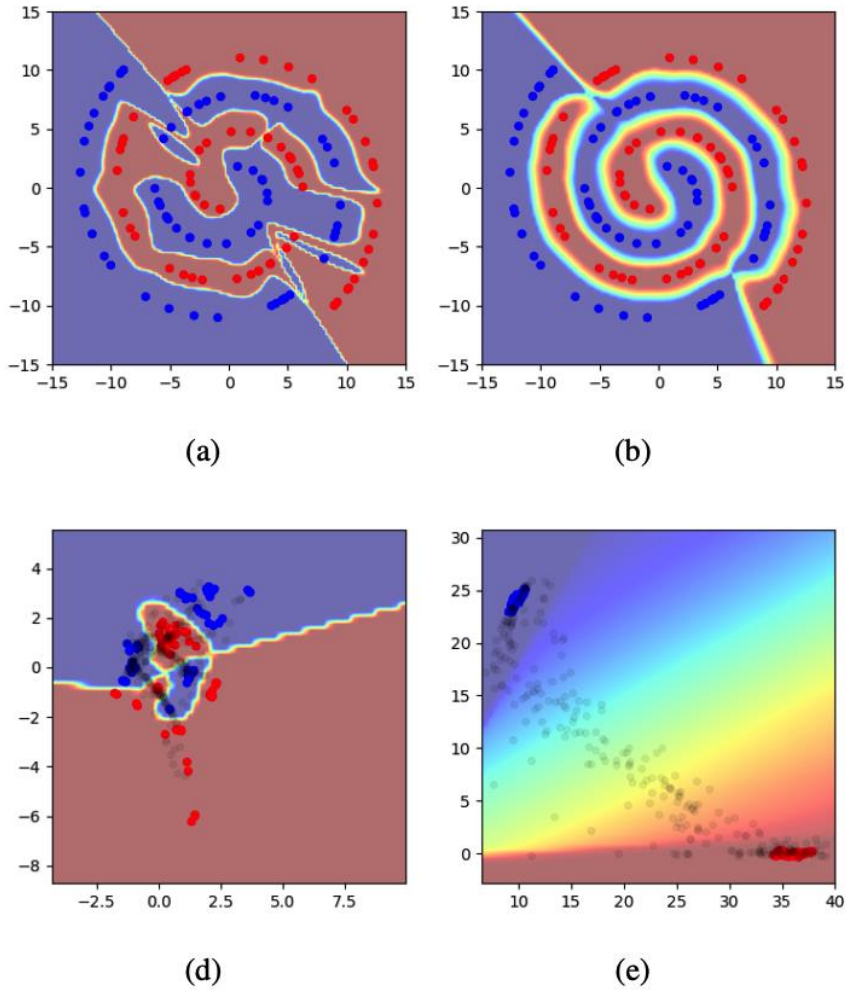
Vicinal Risk Minimization

vicinity distribution

$$\mathrm{d}P_{\mathrm{vic}}(x,y) = \frac{1}{n}\sum_i \nu(\tilde{x},\tilde{y}|x_i,y_i)$$

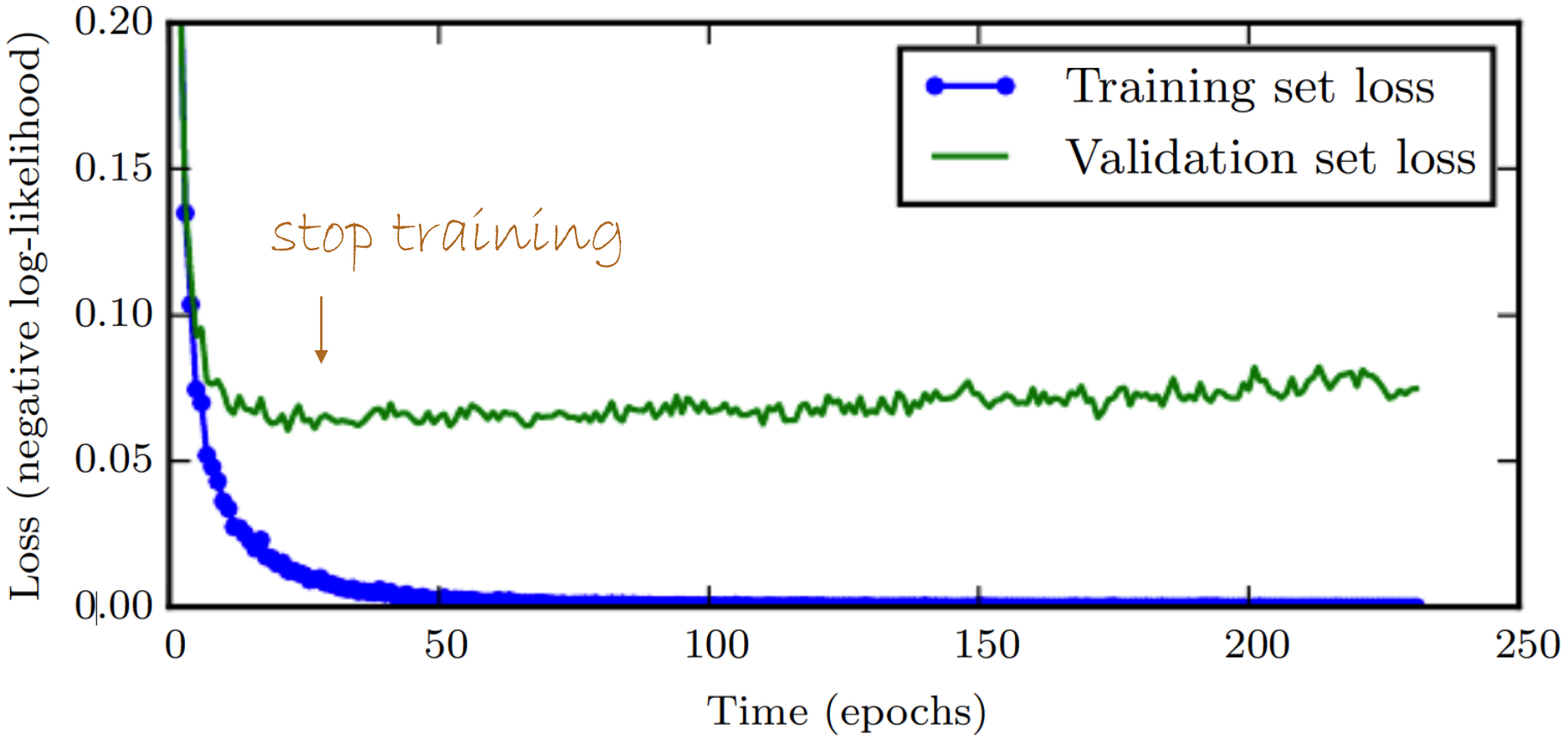
Example: (manifold) Mixup





Example: Additive Noise

Early Stopping



Early Stopping

keep lowest validation;
stop if patience expires

Algorithm 7.1 The early stopping meta-algorithm for determining the best amount of time to train. This meta-algorithm is a general strategy that works well with a variety of training algorithms and ways of quantifying error on the validation set.

Let n be the number of steps between evaluations.
Let p be the "patience," the number of times to observe worsening validation set error before giving up.
Let $\boldsymbol{\theta}_o$ be the initial parameters.
$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta}_o$
$i \leftarrow 0$
$j \leftarrow 0$
$v \leftarrow \infty$
$\boldsymbol{\theta}^* \leftarrow \boldsymbol{\theta}$
$i^* \leftarrow i$
while $j < p$ **do**
  Update $\boldsymbol{\theta}$ by running the training algorithm for n steps.
  $i \leftarrow i + n$
  $v' \leftarrow \text{ValidationSetError}(\boldsymbol{\theta})$
  if $v' < v$ **then**
    $j \leftarrow 0$
    $\boldsymbol{\theta}^* \leftarrow \boldsymbol{\theta}$
    $i^* \leftarrow i$
    $v \leftarrow v'$
  else
    $j \leftarrow j + 1$
  end if
end while
Best parameters are $\boldsymbol{\theta}^*$, best number of training steps is i^*

Early Stopping is Equiv. to Weight Decay

Effective capacity: $\epsilon \cdot \tau$ (learning rate x optimization steps)

The update step

$$\omega^{(\tau)} = \omega^{(\tau-1)} - \epsilon H\left(\omega^{(\tau-1)} - \omega^*\right)$$

From which we obtain

$$Q^T\left(\omega^{(\tau)} - \omega^*\right) = (I - \epsilon\Lambda)Q^T(\omega^{(\tau-1)} - \omega^*)$$

$$H = Q\Lambda Q^T$$

Early Stopping is Equivalent to Weight Decay

For Early Stopping we obtain

$$Q^T \omega^{(\tau)} = [I - (I - \epsilon\Lambda)^\tau] Q^T \omega^*$$

For Weight Decay we had

$$Q^T \tilde{\omega} = [I - (\Lambda + \alpha I)^{-1} \alpha] Q^T \omega^*$$

ES equiv. WD if

$$(I - \epsilon\Lambda)^\tau = (\Lambda + \alpha \mathrm{I})^{-1} \alpha \quad \longrightarrow \quad \alpha \approx \frac{1}{\tau\epsilon}$$

Dropout

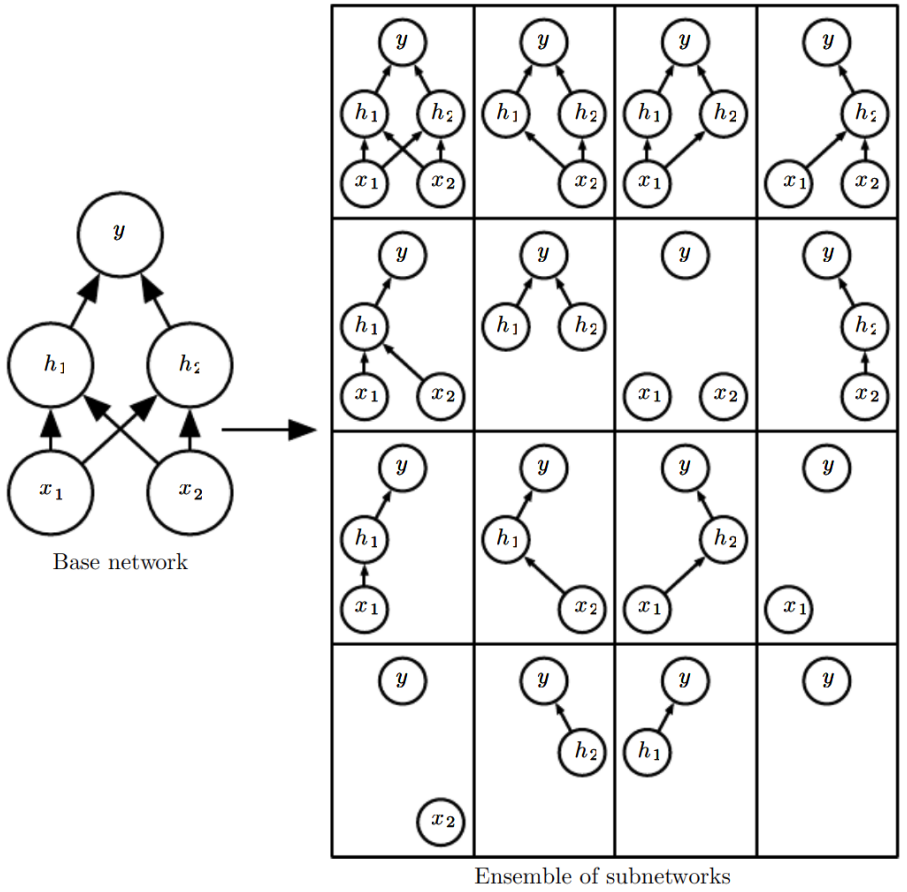
How can we train and evaluate many (many) models?

Bagging:

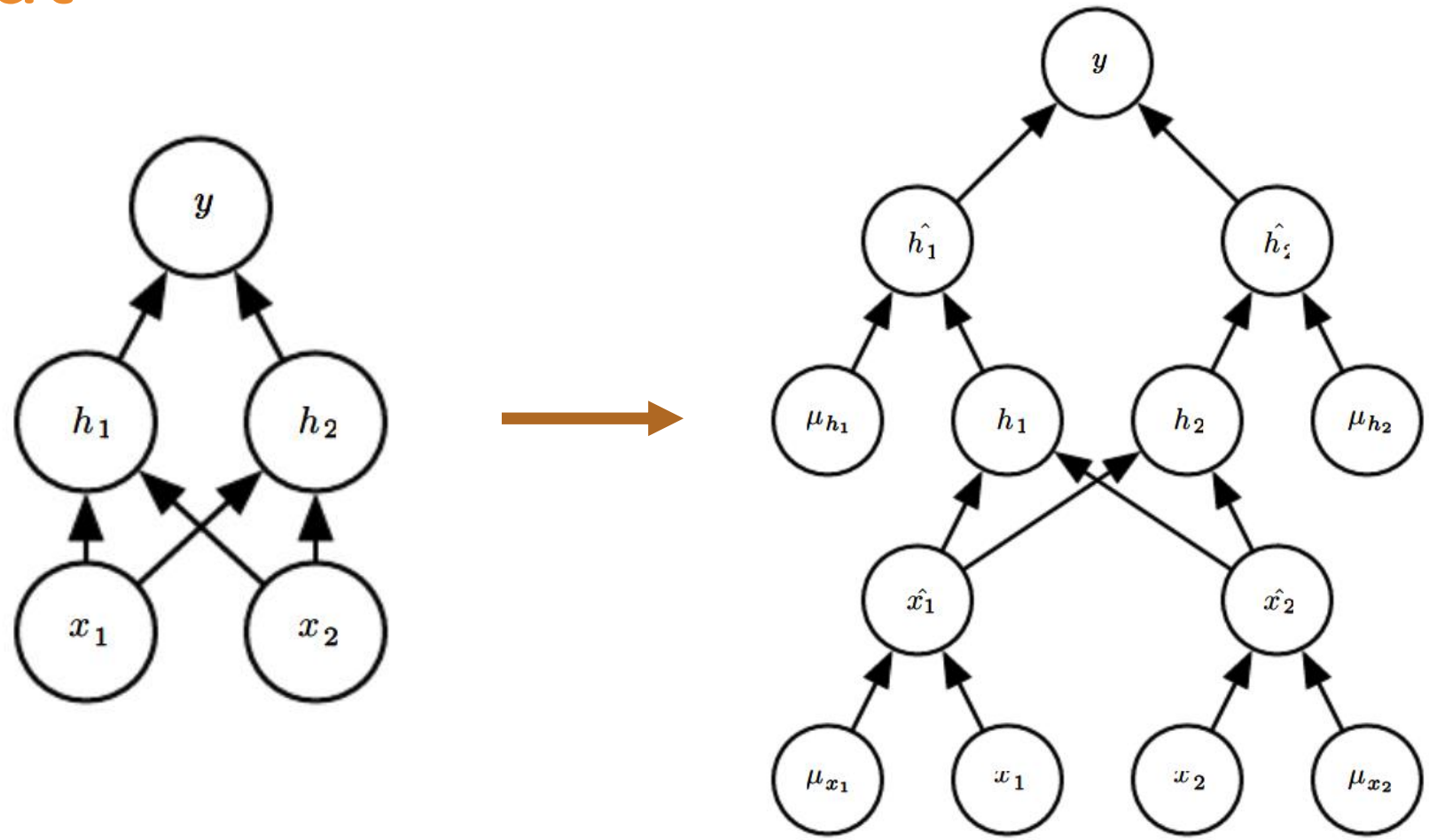
define k models

construct k datasets

train model i on dataset i

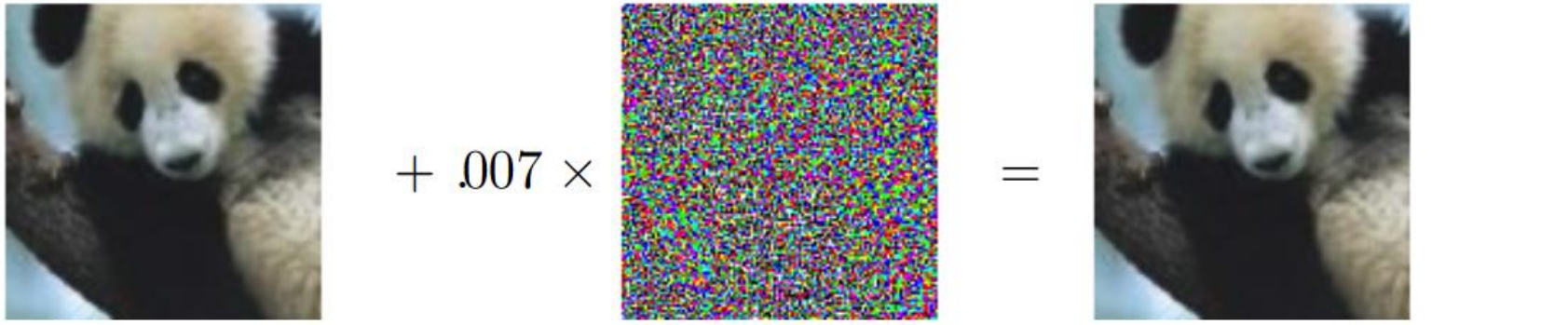


Dropout



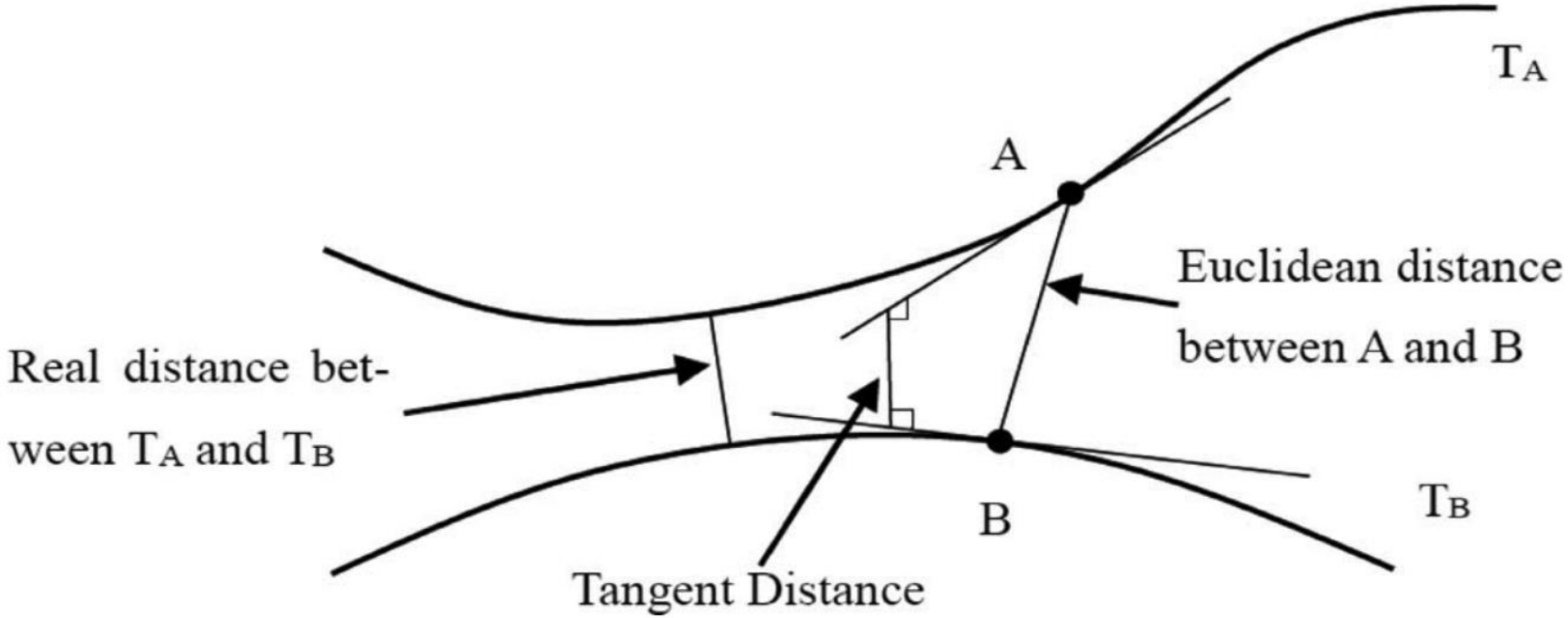
Adversarial Training

Do models truly obtain human-level understanding?



Tangent Distance

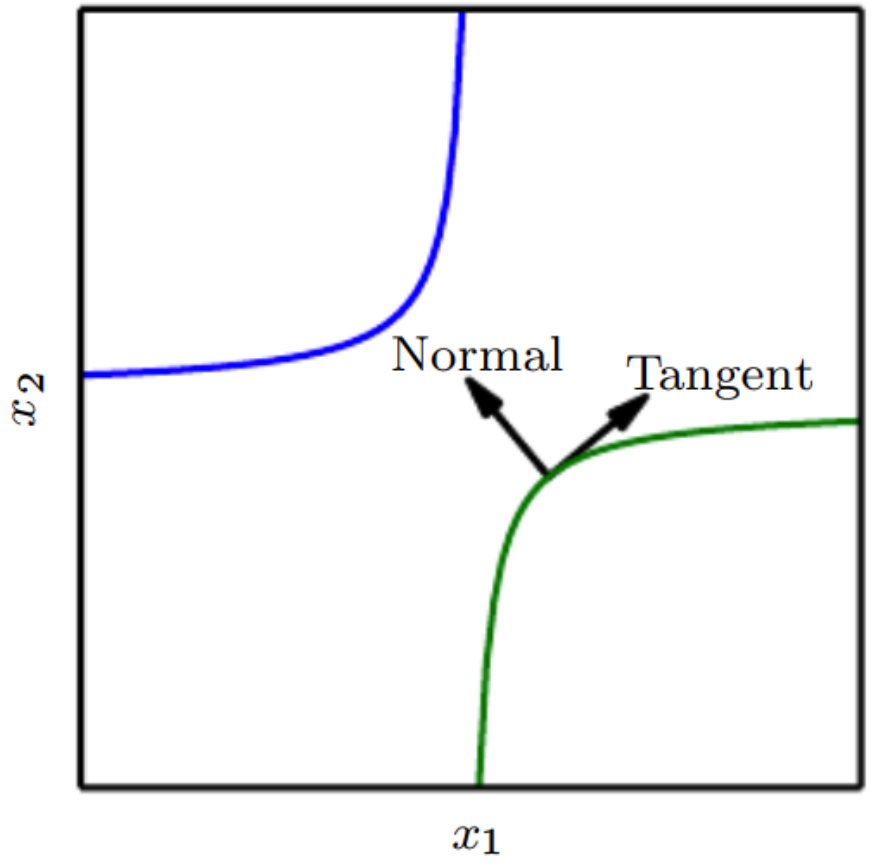
A nearest-neighbor algorithm with a non-Euclidean metric



Tangent Prop

Add a penalty

$$\Omega(f) = \sum_i \left(v^{(i)T} \nabla_x f(x) \right)^2$$



Code Demo